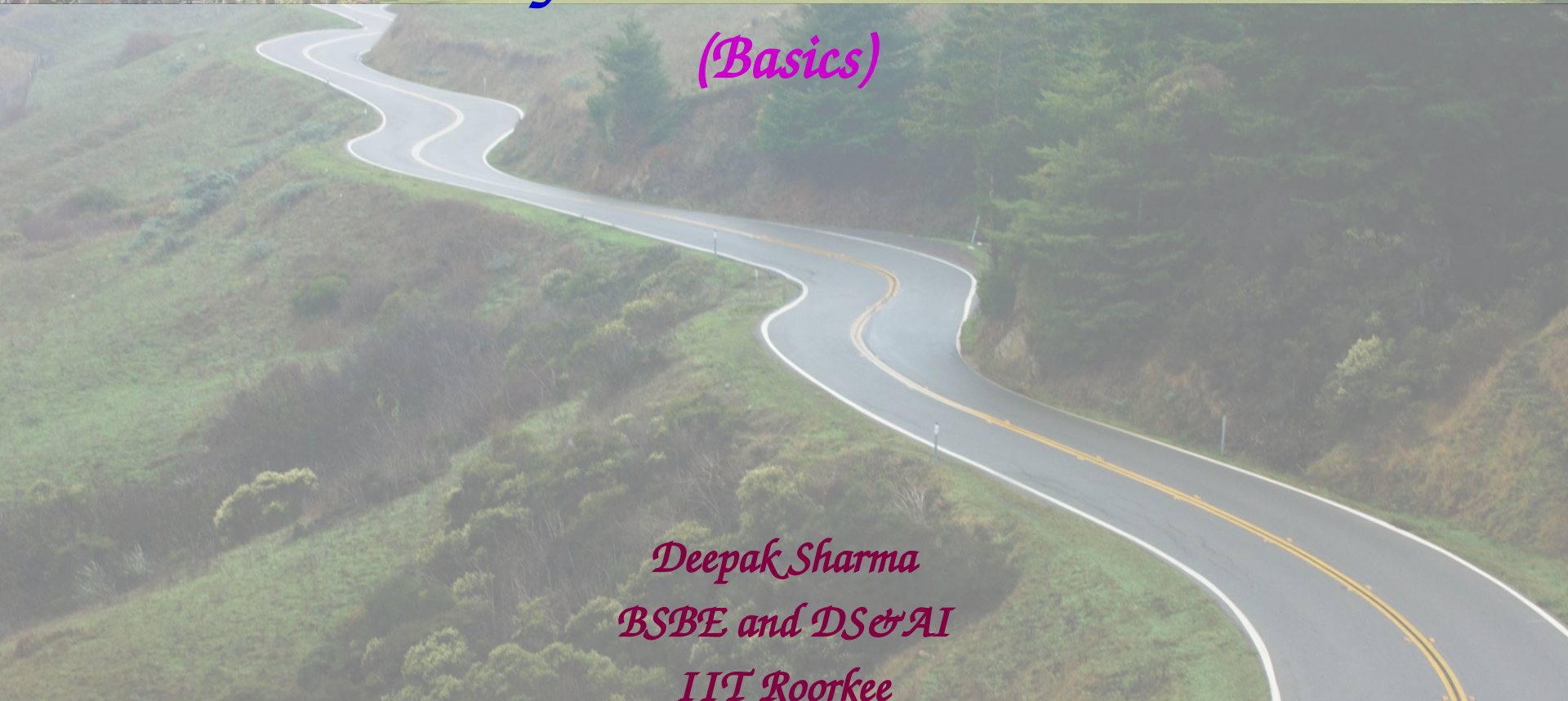


A photograph of a university campus featuring several palm trees in the foreground and a large, multi-story brick building in the background.

Python – Lectures 1 & 2

(Basics)

A photograph of a winding asphalt road that curves through a lush green hillside. The road has white and yellow lane markings.

Deepak Sharma
BSBE and DS&AI
IIT Roorkee



Historical background



- First truly modern computer was *Mark I* (1944).
- In 1936, mathematician Alan Turing described a hypothetical computing device called a *universal Turing machine*.
- The *Church-Turing thesis* states that if a function is computable, a Turing machine can be programmed to compute it.
- A programming language is said to be *Turing complete* if it can be used to simulate a universal Turing machine.
- All modern programming languages are Turing complete.

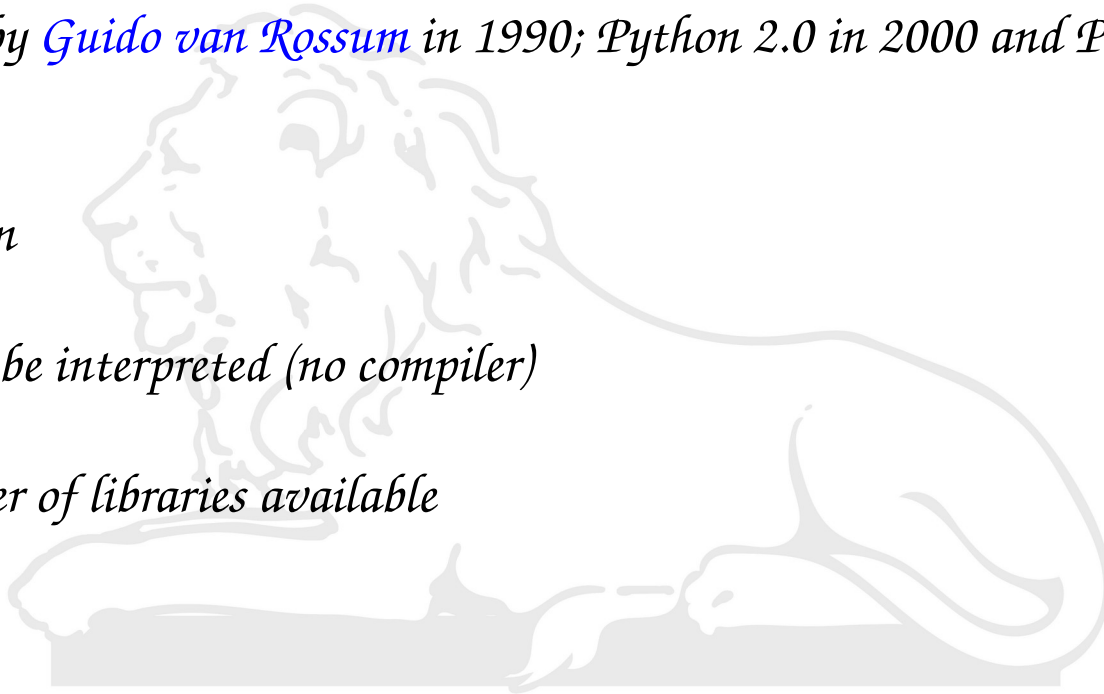
Programming languages

- *Anything that can be programmed in one language (eg. Python) can be programmed in any other language (eg. C++).*
- *However, some things may be easier to program in a particular language:*
 - ***MATLAB**: for manipulating vectors and matrices*
 - ***C**: for programs that control data networks*
 - ***PHP**: for building Web sites*
 - ***SQL**: for building databases*
 - ***Python**: a general purpose language*

Programming rules

- *Primitive constructs (words): literals (eg. number 3.2 or string abc) and infix operators (eg. + or /)*
- *Syntax: defines whether the strings of characters/symbols are well formed (eg. 2.3 2.3 is incorrect)*
- *Static semantics: defines which syntactically valid strings have meaning [eg 3.2/'abc' is syntactically well formed (<literal><operator><literal>) but incorrect]*
- *Semantics: associates meaning with syntactically correct strings with no static semantic errors*

- *General-purpose programming language that can be used effectively to build any kind of program*
- *Introduced by [Guido van Rossum](#) in 1990; Python 2.0 in 2000 and Python 3.0 in 2008*
- *Easy to learn*
- *Designed to be interpreted (no compiler)*
- *Large number of libraries available*



Python installation



```
Python — -bash — 71x19
[Deepaks-MacBook-Pro:Python deepak$ python --version
Python 2.7.10
[Deepaks-MacBook-Pro:Python deepak$ python
Python 2.7.10 (default, Oct 23 2015, 19:19:21)
[GCC 4.2.1 Compatible Apple LLVM 7.0.0 (clang-700.0.59.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> exit()
[Deepaks-MacBook-Pro:Python deepak$
[Deepaks-MacBook-Pro:Python deepak$
[Deepaks-MacBook-Pro:Python deepak$ python3 --version
Python 3.7.3
[Deepaks-MacBook-Pro:Python deepak$ python3
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:52:21)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> exit()
Deepaks-MacBook-Pro:Python deepak$
```

Scalar:

Single value

int

integers

float

real numbers

bool

true/false

None

null

Non-Scalar:

Many objects

string

sequence of symbols (words)

tuple

ordered, immutable

list

ordered, mutable

dict

ordered, mutable, key-value pair

set

unordered, mutable, unique

Operators



Add/Sub

+, -

Mult/Div

**, /*

(// integer/floor division)

Remainder

%

Power

Equality

==, !=

Relational

<, <=, >, >=



First program



Python — vi circle_area.py — 58×15

```
#!/usr/bin/python
```

Command interpretation

```
#####
```

```
#calculate the area of a circle
```

Comment

```
#####
```

```
pi = 3.14
```

```
radius = 11
```

```
area = pi*(radius**2)
```

```
print area
```

```
~
```

```
~
```

```
~
```

```
~
```

```
"circle_area.py" 10L
```

Python — -bash — 62×15

```
Deepaks-MacBook-Pro:Python deepak$ ls -lt
```

```
total 8
```

```
-rw-r--r--  1 deepak  admin  174 Jun 17 13:05 circle_area.py
```

```
Deepaks-MacBook-Pro:Python deepak$
```

```
Deepaks-MacBook-Pro:Python deepak$ python circle_area.py
```

```
379.94
```

```
Deepaks-MacBook-Pro:Python deepak$ chmod a+x circle_area.py
```

```
Deepaks-MacBook-Pro:Python deepak$ ls -lt
```

```
total 8
```

```
-rwxr-xr-x  1 deepak  admin  174 Jun 17 13:05 circle_area.py
```

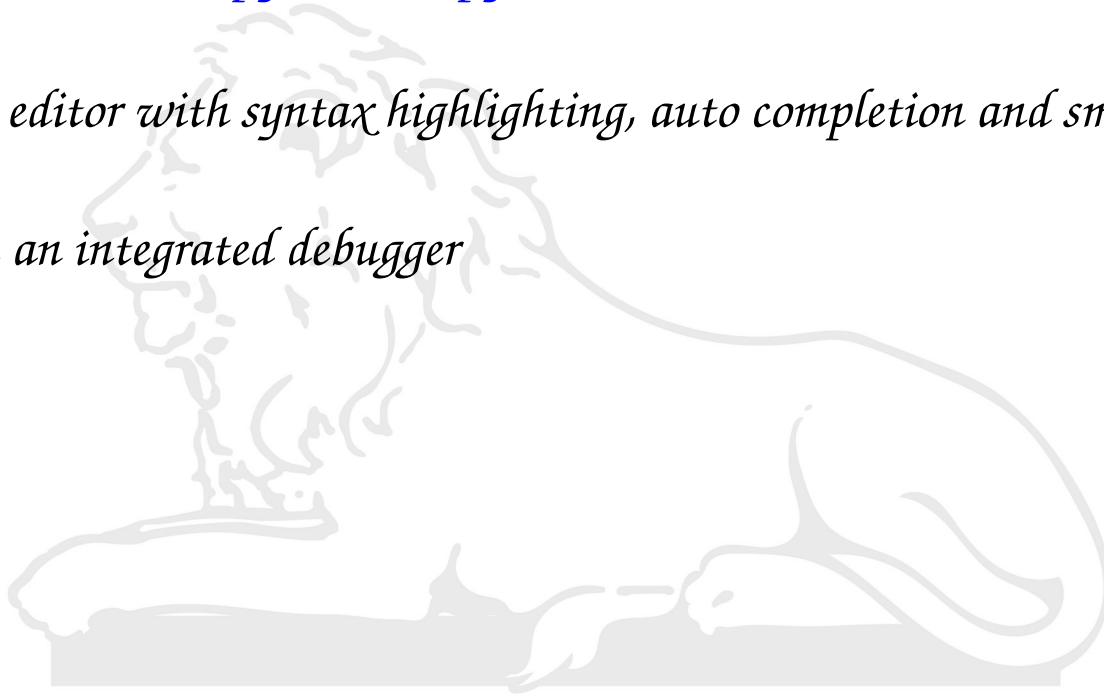
```
Deepaks-MacBook-Pro:Python deepak$
```

```
Deepaks-MacBook-Pro:Python deepak$ ./circle_area.py
```

```
379.94
```

```
Deepaks-MacBook-Pro:Python deepak$
```

- One can also do programming using *integrated development environment (IDE)*
- *PyCharm, Anaconda, Jupyter, Canopy*
- *Provide text editor with syntax highlighting, auto completion and smart indentation*
- *Also provide an integrated debugger*



Float (Jupyter notebook)

```
In [1]: a = 0.1  
        b = 0.2  
        c = a + b  
        c
```

Out[1]: 0.30000000000000004

```
In [2]: c == 0.3
```

Out[2]: False

```
In [3]: from decimal import Decimal  
        d = Decimal("0.1") + Decimal("0.2")  
        d
```

Out[3]: Decimal('0.3')

```
In [4]: print(d)
```

0.3

Float



```
In [1]: from decimal import Decimal  
d = Decimal("0.1") + Decimal("0.2")  
print(d)
```

0.3

```
In [2]: a = 0.3  
d == a
```

Out[2]: False

```
In [3]: print(type(d))  
  
<class 'decimal.Decimal'>
```

```
In [4]: print(type(a))  
  
<class 'float'>
```

```
In [5]: d = float(d)  
d == a
```

Out[5]: True

Boolean



```
In [1]: a=True  
a
```

```
Out[1]: True
```

```
In [2]: type(a)
```

```
Out[2]: bool
```

```
In [3]: b=bool(0)  
b
```

```
Out[3]: False
```


Identity operators

- *is, is not*
- *They compare the memory locations of two objects*

```
In [1]: x = [1, 2, 3]
        y = [1, 2, 3]
```

```
In [2]: print(x == y)

True
```

```
In [3]: print(x is y)    # different objects in memory

False
```

```
In [4]: print(x is not y)

True
```

- *A special constant representing the absence of a value or a null value.*

```
In [1]: a = None  
a is None
```

```
Out[1]: True
```

```
In [2]: b = 1  
b is not None
```

```
Out[2]: True
```

```
In [3]: print(type(None))  
  
<class 'NoneType'>
```

```
In [4]: variable = None  
  
if variable is None:  
    print("variable has a value of None")  
else:  
    print("variable has a value")  
  
variable has a value of None
```

Input, Type and Type conversions

```
Python — vi input_type.py — 87x21
#!/Library/Frameworks/Python.framework/Versions/3.7/bin/python3

#####
#Input, Type and Type conversions
#####

n = input('Enter an integer: ')
print(type(n), '\n')           #To find type of the object

x = n*4
print(x, '\n')

y=int(n)*4
print(y, '\n')

z=int(3.9)
print(z, '\n')
~
~
~
"input_type.py" 17L, 446C
```

```
Python — -bash — 59x12
Deepaks-MacBook-Pro:Python deepak$ ./input_type.py
Enter an integer: 3
<class 'str'>

3333

12

3

Deepaks-MacBook-Pro:Python deepak$
```

Multiple assignment



```
Python — vi multiple_assign.py — 65x15
#!/Library/Frameworks/Python.framework/Versions/3.7/bin/python3

#####
#multiple assignment
#####

x, y = 2, 3
x, y = y, x
print('x =', x)
print('y =', y)
~
~
~
~
```

```
Python — -bash — 65x9
[Deepaks-MacBook-Pro:Python deepak$ ./multiple_assign.py]
x = 3
y = 2
[Deepaks-MacBook-Pro:Python deepak$ python multiple_assign.py]
('x =', 3)
('y =', 2)
Deepaks-MacBook-Pro:Python deepak$
```

Not backward compatible

Overloaded operators

```
Python — -bash — 72×20
Deepaks-MacBook-Pro:Python deepak$ python3
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:52:21)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 3+4
7
>>> 'Deepak'+'Sharma'
'DeepakSharma'
>>>
>>>
>>> 3*4
12
>>> 3*'Repeat'
'RepeatRepeatRepeat'
>>>
>>>
>>> exit()
Deepaks-MacBook-Pro:Python
```

```
AS2024 — IPython: BEC101/AS2024 — -bash — 68×21
(base) Deepaks-MacBook-Pro:AS2024 deepak$ ipython
Python 3.7.6 (default, Jan 8 2020, 13:42:34)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.12.0 -- An enhanced Interactive Python. Type '?' for help.

[In [1]: 3+4
Out[1]: 7

[In [2]: 'Deepak'+'Sharma'
Out[2]: 'DeepakSharma'

[In [3]: 3*4
Out[3]: 12

[In [4]: 3*'Repeat'
Out[4]: 'RepeatRepeatRepeat'

[In [5]: exit()
(base) Deepaks-MacBook-Pro:AS2024 deepak$
```

String



```
In [1]: s1 = 'abc'  
s1
```

```
Out[1]: 'abc'
```

```
In [2]: s2 = '1'  
s2
```

```
Out[2]: '1'
```

```
In [3]: str(1)
```

```
Out[3]: '1'
```



String operations

```
Python — vi string.py — 87×20
#!/Library/Frameworks/Python.framework/Versions/3.7/bin/python3

#####
#String operations
#####

Length = len('abcd')           #Finds length of a string
print(Length, '\n')

Char_first = 'abcd'[0]         #Indexing is done to extract characters from string
Char_last = 'abcd'[-1]
print(Char_first, Char_last)
print(Char_first + Char_last, '\n')

Substring = 'abcd'[0:2]        #Slicing is used to extract substrings of arbitrary lengths
print(Substring)
~
~
~
"string.py" 16L, 505C
```

```
Python — -bash — 49×8
[Deepaks-MacBook-Pro:Python deepak$ ./string.py ]
4
a d
ad
ab
Deepaks-MacBook-Pro:Python deepak$
```


Further reading



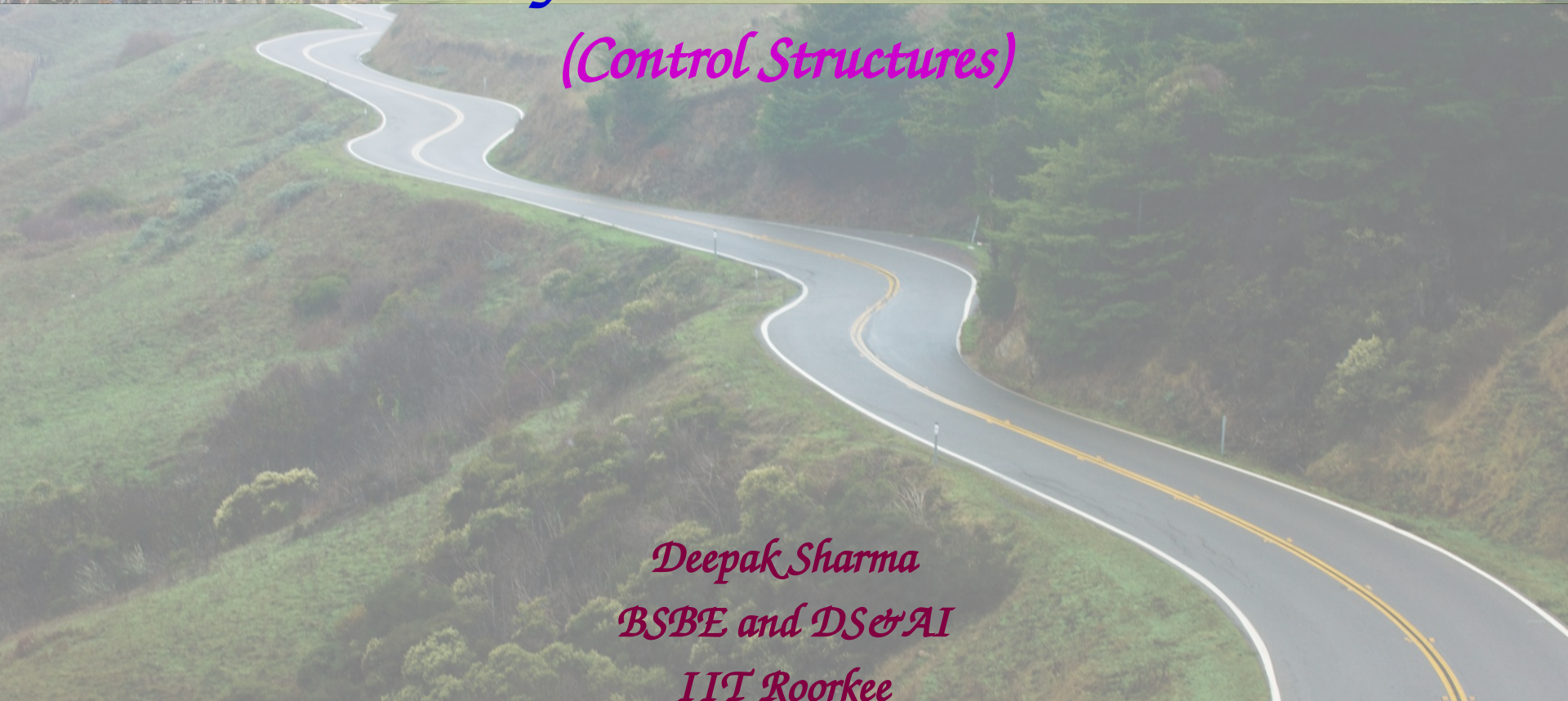
Introduction to Computation and Programming Using Python

With Application to Understanding Data

second edition

John V. Guttag

Python – Lectures 3 & 4
(Control Structures)

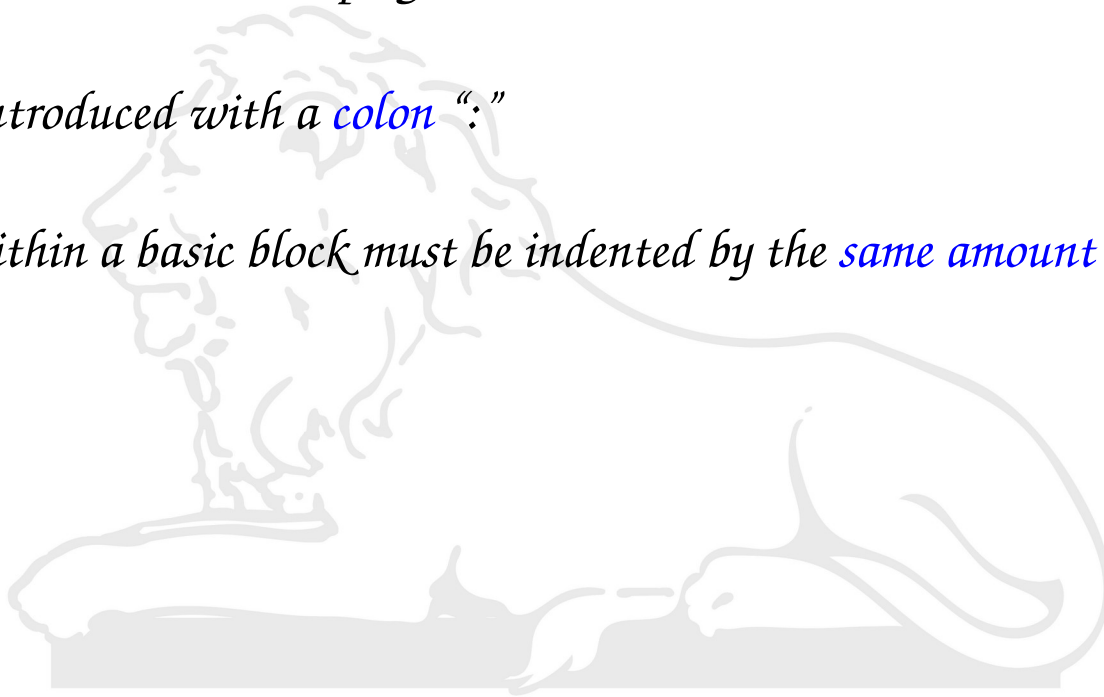


Deepak Sharma
BSBE and DS&AI
IIT Roorkee



Indentation

- In Python, *indentation* is significant
- *Whitespace* is used to delimit program blocks
- Blocks are introduced with a *colon* ":"
- Each line within a basic block must be indented by the *same amount*



if, else and elif statements

```
Python — vi if_else_elif.py — 70x21
#!/Library/Frameworks/Python.framework/Versions/3.7/bin/python3

#####
#if, else and elif
#####

x = input('Enter an integer: ')
x = int(x)

if x%2 == 0:
    if x%3 == 0:
        print ('Divisible by 2 and 3')
    else:
        print ('Divisible by 2 and not by 3')
elif x%3 == 0:
    print('Divisible by 3 and not by 2')
    print('Indented')
print('Not indented')
~
~
"if_else_elif.py" 18L, 400C
```

```
Python — -bash — 53x10
Deepaks-MacBook-Pro:Python deepak$ ./if_else_elif.py
Enter an integer: 6
Divisible by 2 and 3
Not indented
Deepaks-MacBook-Pro:Python deepak$ ./if_else_elif.py
Enter an integer: 9
Divisible by 3 and not by 2
Indented
Not indented
Deepaks-MacBook-Pro:Python deepak$
```

Compound expressions

```
Python — vi compound_cond.py — 70×19
#!/Library/Frameworks/Python.framework/Versions/3.7/bin/python3

#####
#if, elif and compound conditions
#####

x = input('Enter an integer: ')
x = int(x)

if x%2 == 0 and x%3 == 0:
#if x%2 == 0 or x%3 == 0:
    print('Divisible by 2 and 3')
elif x%2 == 0:
    print('Divisible by 2 and not by 3')
elif x%3 == 0:
    print('Divisible b
~
~
"compound_cond.py" 16L, 40
```

```
Python — -bash — 56×7
Deepaks-MacBook-Pro:Python deepak$ ./compound_cond.py
Enter an integer: 6
Divisible by 2 and 3
Deepaks-MacBook-Pro:Python deepak$ ./compound_cond.py
Enter an integer: 9
Divisible by 3 and not by 2
Deepaks-MacBook-Pro:Python deepak$
```

Bitwise operators

- *Bitwise AND (&):* This operator compares corresponding bits of two integers. If both bits at a given position are 1, the resulting bit is 1; otherwise, it is 0.

```
In [1]: a = 5           # 0101 in binary
        b = 3           # 0011 in binary
        result = a & b  # 0001 (1 in decimal)
        print(result)
```

1

Bitwise OR (|): This operator compares corresponding bits of two integers. If at least one of the bits at a given position is 1, the resulting bit is 1; otherwise, it is 0.

```
In [2]: a = 5           # 0101 in binary
        b = 3           # 0011 in binary
        result = a | b  # 0111 (7 in decimal)
        print(result)
```

7